Question 1

You are tasked with developing a program for a university enrollment system where students' enrollment details need to be managed. The system should handle the following tasks:

1. Add Student Details: Add the details of a student (name, enrollmentID, course, year) to the system.

2. Search Student by EnrollmentID: Search and return the details of a student for the given enrollmentID.

3. Validate EnrollmentID: Ensure the enrollmentID follows the format 'ER2024006'. If the format is invalid, throw a custom exception.

4. Get Students by Course: Return all students enrolled in a given course.

5. Get Students by Year: Return all students enrolled in a specific year.

Constraints:

- EnrollmentID must follow the format 'ERYYYYNNN' where 'ER' are constant characters, 'YYYY' represents the enrollment year, and 'NNN' is the candidate's roll number.

- Use custom exception handling to manage invalid enrollmentID formats.

Utilize Java Streams and Lambda expressions for operations.

Classes Involved:

- Student Class:

- A POJO representing a student with attributes: name, enrollmentID, course, and year.

- EnrollmentSystem Class:

- Maintains a list of students.

Implements methods to add a student, search students by enrollmentID, validate enrollmentID, get students by course, and get students by year.

Uses Java Streams and Lambda expressions to perform operations on the student list.

- InvalidEnrollmentIDException Class:

- Custom exception to handle invalid enrollmentID formats.

- UserInterface (Main Class):

Demonstrates adding students, searching by enrollmentID, validating enrollmentID, fetching students by course, and fetching students by year.

Initial Code Skeleton

File Name: Student.java

```
public class Student {
private String name;
private String enrollmentID;
private String course;
private int year;
// Constructor, getters, setters
}
```

File Name: Invalid EnrollmentIDException.java

```
public class Invalid EnrollmentIDException extends Exception {
public Invalid EnrollmentIDException(String message) {
super(message);
}
}
```

File Name: EnrollmentSystem.java

```
public class Enrollment System {
private List<Student> students = new ArrayList<>();
// 1. Add student details
public void addStudent(String name, String enrollmentID, String course, int year) {
}

// 2. Search student by enrollmentID
public Optional<Student> search Student By EnrollmentID (String enrollmentID) {
```

```
}
// 3. Validate enrollmentID
public boolean isValidEnrollmentID(String enrollmentID) {
}
// 4. Get students by course
public List<Student> getStudentsByCourse (String course) {
}
// 5. Get students by year
public List<Student> getStudentsByYear(int year) {
}
}
```

File Name: UserInterface.java

```
Public class UserInterface{
public static void main(String[] args) {
Enrollment System enrollmentSystem = new Enrollment System();
try {
// Adding students
// Search for a student by enrollmentID
// Get students by course
// Get students by year
}
```

Sample Input and Output

Sample Input 1:

Adding Students:

Alice, ER2024001, Computer Science, 2024

Bob, ER2024002, Mechanical Engineering, 2024

Charlie, ER2024003, Electrical Engineering, 2024

Dave, ER2024004, Computer Science, 2024

Eve, ER2024005, Mechanical Engineering, 2024

Search for student by enrollmentID:

ER2024003

Get students by course:

Computer Science

Get students by year:

2024

Sample Output 1:

Search Results for 'ER2024003':

Charlie | ER2024003 | Electrical Engineering | 2024

Students in 'Computer Science':

Alice | ER2024001 | Computer Science | 2024

Dave | ER2024004 | Computer Science | 2024

Students enrolled in the year 2024:

Alice | ER2024001 | Computer Science | 2024

Bob | ER2024002 | Mechanical Engineering | 2024

Charlie | ER2024003 | Electrical Engineering | 2024

Dave | ER2024004 | Computer Science | 2024

Eve | ER2024005 | Mechanical Engineering |

2024

Sample Input 2:

Adding Students:

Fay, ER2024006, Biology, 2024

Grace, ER2024007, Chemistry, 2024

Heidi, ER2024008, Physics, 2024

Ivan, ER2024009, Mathematics, 2024

Judy, ER2024010, Biology, 2024

Search for student by enrollmentID:

ER2024008

Get students by course:

Biology

Get students by year:

2024

Sample Output 2:

Search Results for 'ER2024008':

Heidi | ER2024008 | Physics | 2024

Students in 'Biology':

Fay | ER2024006 | Biology | 2024

Judy | ER2024010 | Biology | 2024

Students enrolled in the year 2024:

Fay | ER2024006 | Biology | 2024

Grace | ER2024007 | Chemistry | 2024

Heidi | ER2024008 | Physics | 2024

Ivan | ER2024009 | Mathematics | 2024

Judy | ER2024010 | Biology | 2024

Question 2

Create an online survey platform for easy survey creation and response collection. Meta surveys, validate details, update status, and send reminders as needed. Use exception handling to prevent crashes from invalid inputs. Parse the input to extract the survey information.

Input format: <survey name>:<visibility>:<number of responses>:<start date>:<end date>.

Component Specification: Invalid Survey Exception (This class inherits the Exception Class)

Type (Class)	**Responsibility**
InvalidSurvey Exception	Provided with a single-argument constructor: **public InvalidSurveyException(String message)**

Validation Rules

- The survey's visibility is case-insensitive and should be one of the three: general, limited, or internal.
- The number of responses should be between 1000 and 5000 (inclusive).
- The start date and end date of the survey should be in the format dd-MM-yyyy.

When the user input is correct and no exceptions are raised, calculate the difference in days between the survey's start and end dates.

- If there is a difference in days, then display the message **"Reminder! only <daysDifference> days left. Fill, if not yet. Ignore, if done".** Otherwise, display **"Inactive survey closed".**
- If the start and end dates are not in the format mentioned above, raise the ParseException with the message **"Invalid date format".**
- If the number of responses is less than the above-mentioned range, then the user-defined exception should be raised with the message **"Survey end date extension needed for low response rate"**, if more than the above-mentioned range, then the user-defined exception should be raised with the message **"High response rate now, more expected by survey end"** respectively.
- If the visibility of the survey is not in the above-mentioned types, then the user-defined exception should be raised with the message **"Invalid visibility".**
- Display **"Thanks for using the application"** at the end, regardless of the number of responses, the start and end dates of the survey, the visibility of the survey, or the difference in days.
- In the UserInterface class, get the string input from the user for validation. Use the exception handling mechanism in the main method to handle the exception. Use a catch block for handling the exceptions. In the catch block, display the appropriate messages.

Note:

- Try, catch, and finally, blocks are mandatory.

- In the sample input and output provided, the highlighted text in bold corresponds to the input given by the user, and the rest of the text represents the output.
- Ensure to follow the object-oriented specifications provided in the question description.
- Ensure to provide the names for classes, attributes, and methods as specified in the question description.
- Adhere to the code template, if provided.
- **Please do not use System.exit(0); to terminate the program.**

Sample Input / Output 1

Enter the survey details:

Product Survey: General:2130:12-05-2022:12-07-2022

Reminder! only 61 days left. Fill, if not yet. Ignore, if done

Thanks for using the application

Sample Input / Output 2

Enter the survey details

Health Survey: General:2130:12-07-2022:12-07-2022

Inactive survey closed

Thanks for using the application

Sample Input / Output 3

Enter the survey details

Product Survey: External:2130:12-07-2022:12-07-2022

Invalid visibility

Thanks for using the application

Sample Input / Output 4

Enter the survey details

Service Survey: General:6000:12-05-2022:23-06-2022

High response rate now, more expected by survey end

Thanks for using the application

Sample Input / Output 5

Enter the survey details

Election Survey: General:200:12-05-2022:23-06-2022

Survey end date extension needed for low response rate

Thanks for using the application

Sample Input / Output 6

Enter the survey details

Fitness Survey: General: 1250:12-05-2022:23/06/2022

Invalid date format

Thanks for using the application

Question 3

NewGen pvt. Ltd. has n number of employees and Olivia who is the HR of the organization needs to maintain the employee details. You need to help her in this activity.

1. Add the employee details (name, designation, location, salary)

2. Search and return the details of an employee for the given name.

3. Return unique (distinct) employee details from the given list.

4. Return the details of the employee earning highest salary.

Conditions:

- There can be more than one employee with same name, designation, location, and salary in the organization.
- Solution should be achieved using streams and lambda concepts.

Classes involved:

Employee class:

- POJO class which represents an employee with attributes: name, designation, location, and salary.

HRManagement class:

- Maintains a list of employees.

- Implements methods to add an employee, search employees by name, get unique employees, and find the highest paid employee.
- Uses Java Streams and Lambda expressions to perform operations on the employee list.

UserInterface (Main class):

- Demonstrates adding employees, searching by name, fetching unique employees, and finding the highest paid employee.

Initial Code skeleton:

//File name: Employee.java

```
public class Employee {
private String name;
private String designation;
private String location;
private double salary;
}
```

//File name: HRManagement.java

```
public class HRManagement {
private List<Employee> employees = new ArrayList<>();
// 1. write code to Add employee details
public void addEmployee (String name, String designation, String location, double salary) {
// 2. write code to Search employee by name
public List<Employee> search EmployeeByName(String name) {
return null;
}
// 3. write code to Get unique employee details
public List<Employee> getUnique Employees() {
return null;
}
// 4. write code to Get employee with highest salary
public Optional<Employee> getHighest Paid Employee() {
```

```
return null;
}
}
```

//File name: UserInterface.java

```
public class UserInterface{
public static void main(String[] args) {
HRManagement hrManagement = new HRManagement();
// write logic to Adding employees
// write logic to Search for an employee by name
// write logic to Get unique employees
// write logic to Get the highest paid employee
}
}
```

Question 4

Michael and his friend are playing a number game. If Michael says a 4-digit number, his friend should reverse the number and expand the number. Finally, it should print the total of all the digits.

If the number is not a 4-digit number, then print **'<<number>> is an invalid number'.**

Note:

- In the Sample Input / Output provided, the highlighted text in bold corresponds to the input given by

the user and the rest of the text represents the output.

- Adhere to the code template, if provided.

- Please don't use System.exit(0) to terminate the program.

Sample Input/Output 1:

Enter the number:

456

456 is an invalid number

Sample Input/Output 2

Enter the number:

5864

Reversed number is 4685

Expanded Form: 4000+600+80+5

Total: 23

Sample Input/Output 3

Enter the number:

3400

Reversed number is 43

Expanded Form: 40+3

Total: 7

Question 5

James, who regularly creates and publishes new apps on the Play Store, wants to keep track of apps that receive low ratings so she can focus on improving their features. Her manager has asked for assistance in developing a Java program to facilitate this process. Your task, as a software developer, is to create a Java program that fulfills this requirement.

Component Specification: AppInfo

Type (Class)	Attributes	Methods
AppInfo	private Map<String,Double> appMap	Getter and setter methods for the attribute are included in the code skeleton.

Note: In the appMap, appName serves as the key and rating as the value.

Requirement 1: Add the appName and the rating to the Map.

Type (Class)	Methods	Responsibilities
AppInfo	public void **addAppDetails**(String appName, double rating)	This method should add the appName and rating passed as an argument into the appMap.

Requirement 2: Find the rating of the given app and return the same

Type (Class)	Methods	Responsibilities
AppInfo	public double **findAppRating**(String appName)	This method takes the appName as an argument. If the appName is found in the appMap, it should return the rating associated with that appName. Otherwise, it should return -1. **condition: appName is case-sensitive**

Requirement 3: Filter the apps with low rating and return the list of those appName's.zs

Type (Class)	Methods	Responsibilities
AppInfo	public List<String> **findAppsWithLowRating**()	This method is responsible for filtering apps with low ratings and returning a list of their appName's. **Condition: apps with ratings less than or equal to 3 are considered to have low ratings.**

In the UserInterface class, prompt the user to input the number of app details they wish to add. Then, gather the app details from the user in the format specified in the sample input. Split the string by the specified delimiter and call the **addAppDetails()** method to add the app details.

Next, call the **findAppRating()** method to retrieve the rating of the specified appName.

If the rating of the specified appName is not available, print **"<appName> is an invalid app name".**

Invoke the **findApps WithLowRating()** method to get the list of appName with ratings less than or equal to 3.

If there are no appName available with ratings less than or equal to 3, print **"No apps were found with low ratings".**

Note:

- In the Sample Input / Output provided, the highlighted text in bold corresponds to the input given by the user, and the rest of the text represents the output.
- Ensure to follow the object-oriented specifications provided in the question description.

- Ensure to provide the names for the classes, attributes, and methods as specified in the question description.
- Adhere to the code template, if provided.

Sample Input/Output 1:

Enter the number of records to be added:

5

Enter the app details (appName: rating):

YouTube:4

Udemy:4.6

TrackMe:2.7

FaceBook:2.9

WhatsApp:3.6

Enter the app name to search for:

facebook

The rating of the Picsart app is 2.9

Apps with low ratings:

TrackMe

facebook

Sample Input/Output 2:

Enter the number of records to be added:

3

Enter the app details (appName: rating):

Spotify:4

Shadow Fight Two:4.7

Practo:3.8

Enter the app name to search for:

Dream11

Dream11 is an invalid app name

No apps were found with low ratings

Questions on Collections

1. PROBLEM STATEMENT

XYZ Online Bookstore is revamping its user authentication and account management system to improve security and user experience. As a software developer assigned to this project, you are tasked with implementing a User class and an Account Manager class based on the following requirements:

User Class

- Attributes:

- 'username': A string representing the username of the user.
- 'password': A string representing the password of the user.
- `email`: A string representing the email address of the user.
- Create getter and setter methods for all attributes.
- Implement a default constructor that initializes the 'username', 'password', and `email` attributes to empty strings.

Account Manager Class

- Attributes:

- `userList: A list to store instances of the User class.
- Create getter and setter methods for 'userList`.
- Implement a default constructor that initializes 'userList' as an empty list.

Requirements:

1. **registerUser(String username, String password, String email):**

- Method to register a new user.

- Check if the 'username' and 'email' are unique. If either is already taken, return 'false' and do not register the user.

- If both 'username` and `email` are unique, create a new User instance with the provided details, add it to 'userList', and return `true`.

2. **login(String username, String password):**

- Method to authenticate a user's login attempt.

- Check if the 'username' exists in 'userList' and if the corresponding 'password' matches the provided password.

- If valid, return the User object associated with the username. If not valid, return 'null'.

3. **resetPassword(String username, String newPassword):**

Method to reset a user's password.

- Find the user in `userList` by `username` and update the 'password' to 'new Password`.

- Return `true` if the password reset was successful, or 'false' if the username was not found in 'userList`.

4. **updateEmail(String username, String newEmail):**

Method to update a user's email address.

- Find the user in `userList' by 'username' and update the 'email' to 'newEmail`.

- Return `true` if the email update was successful, or 'false` if the username was not found in `userList`.

****Additional Functionality:****

- Implement a method 'getUser By Username(String username)` in AccountManager to retrieve a User object by username from `userList`.

- Implement a method 'deleteUser(String username)` in Account Manager to remove a user from `userList' by username.

****Main Program Interaction:****

The program interacts with the user as follows:

- Prompt the user to choose an action (register, login, reset password, update email, delete user).

- Depending on the action:

- For registration, prompt for 'username', 'password', and 'email'.

- For login, prompt for 'username' and 'password' and display whether the login was successful.

- For password reset, prompt for 'username' and 'new Password and display whether the password was successfully reset.

- For email update, prompt for 'username' and 'newEmail' and display whether the email was successfully updated.

- For user deletion, prompt for 'username' and confirm if the user should be deleted from `userList.

****Sample Input/Output:****

Enter action (register, login, reset password, update email, delete user): register

Enter username: alice

Enter password: pass123

Enter email: alice@example.com

User registered successfully.

Enter action (register, login, reset password, update email, delete user): login

Enter username: alice

Enter password: pass123

Login successful. Welcome, Alice!

Enter action (register, login, reset password, update email, delete user): reset password

Enter username: alice

Enter new password: newpass456

Password reset successful.

Enter action (register, login, reset password, update email, delete user): update email

Enter username: alice

Enter new email: newalice@example.com

Email updated successfully.

Enter action (register, login, reset password, update email, delete user): delete user

Enter username: alice

Are you sure you want to delete user 'alice'? (yes/no): yes

User 'alice' deleted successfully.

Enter action (register, login, reset password, update email, delete user): login

Enter username: alice

Enter password: newpass456

Invalid credentials. Please try again.

2. PROBLEM STATEMENT

ABC Electronics is a leading retailer specializing in electronic gadgets. The company wants to enhance its inventory management system to efficiently track sales based on product categories. The IT department has assigned you, a software developer, to implement a program based on the requirements.

Sales Tracker Class

salesSet =>A set to store sales records in the format of 'customerName:product' pairs. And create setter and getter and default constructor.

SalesTracker => Constructor that initializes an empty 'salesSet`.

Requirement 1: addSalesRecord(String record) => Method to add a sales record to 'salesSet`. Each record is provided in the format 'customerName:product`.

Requirement 2: findNumberOfCustomersByProduct(product) =>Method to count the number of unique customer names who purchased a specific product. If no customers are found for the given product, return `-1`.

Requirement 3: getCustomersByProduct(product) => Method to retrieve a list of customer names

who purchased a specific product. If no customers are found for the given product, return an empty list.

Requirement 4: getProductByCustomer (customerName) => Method to find and return the product purchased by a specific customer. If no product is found for the given customer, return 'None`.

In the **'MAIN'** class, the program interacts with the user as follows:

- Prompt the user to enter the number of sales records they want to add.
- Collect the sales records from the user in the format 'Customer Name: Product`.
- Split the input string by the delimiter and invoke the 'addSales Record' method to add the sales records. After adding records:
- Prompt the user to enter a product name to find the number of customers who purchased it.
- Display the count of customers or print "No customers found for <product>"` if no customers are found.
- Prompt the user to enter a product name to retrieve the list of customers who purchased it.
- Display the list of customers or print "No customers found for <product>"` if no customers are found.
- Prompt the user to enter a customer name to find the product purchased by that customer.
- Display the product purchased by the customer or print "No product found for <customer name>" if no product is found.

Sample Input/Output:

Input/Output 1:

Enter number of records to be added

Enter the records (Customer Name:Product)

Alice:Laptop

Bob: Monitor

Charlie: Keyboard

Enter the Product to be searched

laptop

The number of customers who purchased laptop is 1

Enter the Product to identify the Customer Names

Monitor

Customer names who purchased Monitor are

Bob

Enter the Customer name to find the Product purchased

Charlie

Product purchased by Charlie is Keyboard

Input/Output 2:

Enter number of records to be added

2

Enter the records (Customer Name: Product)

Eve:Headphones

Frank:Tablet

Enter the Product to be searched

smartphone

No customers found for smartphone

Enter the Product to identify the Customer Names

Headphones

Customer names who purchased Headphones are

Eve

Enter the Customer name to find the Product purchased.

adam

No product found for adam

Input/Output 3:

Enter number of records to be added

4

Enter the records (Customer Name:Product)

Alice:Phone

Bob:TV

Charlie: Speaker

David:Phone

Enter the Product to be searched

TV

The number of customers who purchased TV is 1

Enter the Product to identify the Customer Names

Phone

Customer names who purchased Phone are

Alice, David

Enter the Customer name to find the Product purchased

carl

No product found for carl

Questions On Exception Handling

1. SAMPLE -PROBLEM STATEMENT

"SmartHealth, a healthcare management platform, provides an interface for patients to input their health information. This system ensures the accuracy and security of the data entered. Upon validation, the system calculates health insurance premiums based on the user inputs.Robust exception handling is implemented to manage any erroneous inputs effectively.

Consider the following scenario:

You receive input including the patient's name, age, gender, height, weight, blood pressure, and cholesterol levels in the UserInterface class.

Component Specification: **Health Data Processor (Utility class)**

Type (Class)	Method	Parameters	Responsibilities
HealthDataProcessor	validatePatientDetails	int age, String gender, double height, double weight, int bloodPressure, double cholesterol	This method validates the patient details according to specified rules. If all details are valid, it returns true. If any detail is invalid, it throws an InvalidHealthDataException with the appropriate error message.

Validation Rules:

- **Age:** Must be between 18 and 100 inclusive. If not, throw an Invalid HealthData Exception with the message "Invalid age".
- **Gender:** Must be either 'Male' or 'Female' (case sensitive). If not, throw an Invalid Health Data Exception with the message "Invalid gender".
- **Height:** Must be greater than 0. If not, throw an Invalid Health Data Exception with the message "Invalid height".
- **Weight:** Must be greater than 0. If not, throw an Invalid Health Data Exception with the message "Invalid weight".
- **Blood Pressure:** Must be within the range of 60 to 220 mmHg. If not, throw an Invalid HealthData Exception with the message "Invalid blood pressure".
- **Cholesterol:** Must be greater than or equal to 0. If not, throw an Invalid Health Data Exception with the message "Invalid cholesterol".

HealthDataProcessor calculateInsurance Premium double height, double weight, int age, int bloodPressure, double cholesterol This method calculates and returns the health insurance premium based on the patient's health metrics. Guidelines for premium calculation are based on BMI, blood pressure, and cholesterol levels, with specific rates for different age groups and genders.

Component Specification: **InvalidHealthDataException** (This class inherits the Exception

Class)

Type (Class)	Responsibilities
InvalidHealthDataException	Provided with a single-argument constructor: public InvalidHealthDataException(String message). It is thrown when age, gender, height, weight, blood pressure, or cholesterol levels do not follow the validation rules.

The main method in the UserInterface class starts by getting the patient's name, age, gender, height, weight, blood pressure, and cholesterol levels, and then calls the validate Patient

Details() method to validate these details. If the patient details are valid, the method calculates the insurance premium using the calculateInsurance Premium() method and prints the result. If any validation fails, an Invalid Health Data Exception is caught, and the exception message is displayed.

Note:

- Propagate the exceptions that occur in the Health Data Processor class and handle them in the main method.
- In the sample input and output provided, the highlighted text in bold corresponds to the input given by the user, and the rest of the text represents the output.

calculateInsurance Premium() method (take some sample value and calculate the premium)

Example

if height, weight, bloodpressue and cholesterol very high, premium 1500

if height, weight, bloodpressue and cholesterol high, premium 1000

if height, weight, bloodpressue and cholesterol normal, premium 750

Sample Input/output 1:

Enter Name

Emma

Enter Age

25

Enter Gender

Female

Enter Height (in meters)

1.65

Enter Weight (in kg)

60

Enter Blood Pressure (mmHg)

120

Enter Cholesterol Level

5.2

Total Insurance Premium: $750.0

Sample Input/output 2:

Enter Name

John

Enter Age

16

Invalid age

2. SAMPLE -PROBLEM STATEMENT

Banking Application

The banking application allows customers to manage their accounts online. Customers can log in using their credentials, perform transactions, and check their account balance. The system validates inputs to ensure security and correctness. Exception handling is used to manage errors effectively.

Component Specifications: Account Manager (Utility class)

Type (Class)	Method	Parameters	Responsibilities
AccountManager	validateCredentials	String username, String password	Validates the customer's username and password. Throws InvalidCredentialsException if invalid.
AccountManager	deposit	String username, double amount	Deposits the specified amount into the customer's account.
AccountManager	withdraw	String username, double amount	Withdraws the specified amount from the customer's account. Throws InsufficientFundsException if balance is insufficient.
AccountManager	checkBalance	String username	Returns the current balance of the customer's account.

UserInterface Class (Main method)

The main method in the 'UserInterface' class prompts the user to enter their username and password to log in. If the credentials are valid, it allows the user to choose from options to deposit, withdraw, or check balance. Each action calls the appropriate method from Account Manager' to perform operations on the account. Exceptions are handled to provide meaningful error messages to the user.

Additional Validation Conditions

1. Username and Password Validation:

- Username must be at least 6 characters long and can only contain alphanumeric characters and underscores.

- Password must be at least 8 characters long and should include at least one uppercase letter, one lowercase letter, one digit, and one special character.

2. Transaction Amount Validation:

- Deposit amount should be greater than zero and less than or equal to a predefined maximum deposit limit (e.g., $10,000).

- Withdrawal amount should be greater than zero and less than or equal to the current balance.

Explanation:

In this scenario, the 'Account Manager` class handles operations related to account management such as validating credentials, depositing money, withdrawing money, and checking balances. Exceptions (InvalidCredentials Exception' and 'Insufficient FundsException') are used to handle errors gracefully and provide informative messages to the user via the 'UserInterface` class.

Here's an expanded version with additional validation conditions and a more detailed set of sample inputs and outputs for the banking application scenario:

Additional Validation Conditions

1. Username and Password Validation:

- Username must be at least 6 characters long and can only contain alphanumeric characters and underscores.

- Password must be at least 8 characters long and should include at least one uppercase letter, one lowercase letter, one digit, and one special character.

2. Transaction Amount Validation:

- Deposit amount should be greater than zero and less than or equal to a predefined maximum deposit limit (e.g., $10,000).

- Withdrawal amount should be greater than zero and less than or equal to the current balance.

In the provided samples:

Sample 1: demonstrates typical usage where deposits, withdrawals, and balance checks are performed within valid limits.

Sample 2: includes additional validations:

- It handles negative deposit attempts.

- It prevents deposits exceeding a predefined maximum.

- It handles withdrawal attempts that exceed the current balance.

These samples handling of user inputs and error conditions through the 'Account Manager` and 'UserInterface` classes. Each operation checks for validity and ensures that the user receives meaningful feedback through exceptions and informative messages.

By implementing these additional validation checks and error scenarios, the banking application becomes more secure and user-friendly, ensuring that transactions are processed safely and accurately.

Sample Input/Output

Sample Input 1:

Enter Username:

john_doe

Enter Password:

Password123!

Choose an Option:

1. Deposit

2. Withdraw

3. Check Balance

4. Exit

Enter Option:

1

Enter Amount to Deposit:

500

Deposit Successful. New Balance: $1500.00

Enter Option:

2

Enter Amount to Withdraw:

2000

Insufficient Funds. Current Balance: $1500.00

Enter Option:

3

Current Balance: $1500.00

Enter Option:

4

Logout Successful.

Sample Input 2:

Enter Username:

user123

Enter Password:

pass456!

Choose an Option:

1. Deposit

2. Withdraw

3. Check Balance

4. Exit

Enter Option:

1

Enter Amount to Deposit:

-100

Invalid amount. Please enter a positive number.

Enter Amount to Deposit:

15000

Deposit amount exceeds maximum limit of $10,000. Please enter a smaller amount.

Enter Amount to Deposit:

2000

Deposit Successful. New Balance: $2000.00

Enter Option: 3

Current Balance: $2000.00

Enter Option: 2

Enter Amount to Withdraw: 3000

Insufficient Funds. Current Balance: $2000.00

Enter Option:

Logout Successful.

3. SAMPLE-PROBLEM STATEMENT

BookMyHotel is an online platform where users can book hotel rooms based on their preferences and dates of stay. The system requires users to log in using their credentials and validates their details before proceeding with the booking. Upon successful login and validation, the system calculates the total booking cost based on the selected hotel, room

type, duration of stay, and any additional preferences. Exception handling is implemented to manage any invalid inputs effectively.

Consider the following scenario:

You receive input that includes the user's name, age, email, and credit card details in the 'UserInterface' class.

Component Specification: **HotelBooking** (Utility class)

Method: 'validateUserDetails'

Parameters: `int age, String email, String creditCard

Responsibilities: Validates user details according to predefined rules. Throws InvalidBooking Details Exception' with appropriate messages if any detail is invalid.

Validation Rules:

- Age: Must be between 18 and 100 (inclusive). If not, throw an `Invalid Booking Details Exception' with the message "Invalid age".

Email: Must be a valid email format (e.g., name@example.com). If not, throw an InvalidBooking Details Exception' with the message "Invalid email".

- Credit Card: Must be exactly 16 digits. If not, throw an 'Invalid Booking Details Exception with the message "Invalid credit card number".

Method: validate Booking Details

Parameters: String hotelName, String roomType, LocalDate checkinDate, LocalDate

checkOutDate`

Responsibilities: Validates booking details. Throws Invalid Booking Details Exception' with appropriate messages if any detail is invalid.

Validation Rules:

- Hotel Name: Must be a recognized hotel name in the system. If not, throw an Invalid Booking Details Exception' with the message "Invalid hotel name".

- Room Type: Must be either "Standard", "Deluxe", or "Suite" (case sensitive). If not, throw an 'InvalidBooking Details Exception' with the message "Invalid room type".

- Check-in Date and Check-out-Date not in yyyy-MM-dd format throw an Invalid Booking Details Exception' with the message "Invalid date format". (practice with DateTimerFormater(java8) and simpledateformate class)

- Check-in Date: Must be in the future. If not, throw an 'Invalid Booking Details Exception with the message "Invalid check-in date".

- Check-out Date: Must be after the check-in date. If not, throw an Invalid Booking Details Exception' with the message "Invalid check-out date".

Method: 'calculate Booking Cost

Parameters: String roomType, int numNights, boolean breakfastIncluded

Responsibilities: Calculates and returns the total booking cost based on the room type, duration of stay, and whether breakfast is included. Guidelines for cost calculation are based on provided rates for different room types and optional services.

Example (you can modify the below values according to the scenario)

Standard => 1000 per day

Deluxe=>2000 Per day

Suite =>3000 Per day

Breakfast =>100

Component Specification**: Invalid Booking Details Exception**

Responsibilities: Inherits from `Exception'. It has a single-argument constructor: `public Invalid Booking Details Exception(String message). Thrown when any user or booking detail does not follow the validation rules.

The main method in the 'UserInterface' class starts by getting the user's name, age, email, and credit card details, then calls 'validateUserDetails()` to validate these details. If user details are valid, it proceeds to get the hotel name, room type, check-in, and check-out dates, validating these inputs using 'validateBooking Details()`. If all validations pass, the method calculates the total booking cost using 'calculateBooking Cost() and prints the result. If any validation fails, an 'Invalid Booking Details Exception' is caught, and the exception message is displayed.

Sample Input/Output:

Enter Name

John Doe

Enter Age

25

Enter Email

johndoe@example.com

Enter Credit Card Number

1234567890123456

Enter Hotel Name

Hilton

Select Room Type

Suite

Enter Check-in Date (yyyy-MM-dd)

2024-07-01

Enter Check-out Date (yyyy-MM-dd)

2024-07-05

Total Booking Cost: $3000.00

Enter Name

Jane Smith

Enter Age

17

Invalid age

Enter Name

Mike Johnson

Enter Age

30

Enter Email

mikejohnson@example

Invalid email

Enter Name

Sarah Lee

Enter Age

28

Enter Email

sarahlee@example.com

Enter Credit Card Number

12345

Invalid credit card number

Enter Name

Chris Brown

Enter Age

35

Enter Email

chrisbrown@example.com

Enter Credit Card Number

9876543210123456

Enter Hotel Name

Hyatt

Select Room Type

Penthouse

Invalid room type

...

Enter Name

Ella Davis

Enter Age

40

Enter Email

elladavis@example.com

Enter Credit Card Number

1111222233334444

Enter Hotel Name

Marriott

Select Room Type

Standard

Enter Check-in Date (yyyy-MM-dd)

2024-06-27

Enter Check-out Date (yyyy-MM-dd)

2024-06-25

Invalid check-out date

4. SAMPLE -PROBLEM STATEMENT

Healthcare Appointment Booking System

Healthcare Hub, a leading healthcare provider, offers an online portal where patients can schedule appointments by providing their details. The system validates these details according to predefined rules to ensure accurate booking and patient information security. If the booking details are valid, the system schedules the appointment and confirms it.

Exception handling is incorporated to manage invalid inputs effectively.

Scenario Description:

You receive input that includes the patient's name, age, phone number, health insurance ID, doctor's specialization, and appointment date/time in the 'UserInterface' class.

Component Specifications: Appointment Booking System (Utility class)

Type (Class)	Method	Parameters	Responsibilities
AppointmentBookingSystem	alidatePatientDetails	int age, String phoneNumber, String insuranceId	Validates patient details (age, phone number, insurance ID) according to specified rules. Throws `InvalidAppointmentException` if any detail is invalid.
AppointmentBookingSystem	validateDoctorAvailability	String doctorSpecialization, LocalDateTime appointmentDateTime	Checks if the doctor specializing in the given field is available at the specified date and time. Throws `UnavailableDoctorException` if the doctor is not available.
AppointmentBookingSystem	scheduleAppointment	String patientName, String doctorSpecialization, LocalDateTime appointmentDateTime	Books the appointment for the patient with the specified doctor at the given date and time.

Validation Rules:

1. Patient Details:

- **Age**: Must be between 1 and 120. If not, throw 'InvalidAppointmentException' with the message "Invalid age".
- **Phone Number:** Must be exactly 10 digits. If not, throw InvalidAppointmentException' with the message "Invalid phone number".
- **Insurance ID:** Must be alphanumeric. If not, throw 'InvalidAppointment Exception` with the message "Invalid insurance ID".

2. Doctor Availability:

- **Doctor Specialization:** Must match a predefined list of specialties (e.g., "Cardiology", "Dermatology", "Pediatrics"). If not, throw

`UnavailableDoctorException' with the message "Doctor specialization not available".

- **Appointment Date/Time:** Must be in the future and within working hours. If not available, throw `Unavailable DoctorException' with the message "Doctor not available at specified time".

Sample Input/Output:

Sample Input/Output 1:

Enter patient name:

John Doe

Enter age:

45

Enter phone number:

9876543210

Enter insurance ID:

ABCD123456

Enter doctor specialization:

Cardiology

Enter appointment date/time (yyyy-MM-dd HH:mm):

2024-07-10 14:30

Appointment scheduled successfully with Dr. Smith (Cardiology) on 10th July 2024 at 2:30 PM.

Sample Input/Output 2:

Enter patient name:

Alice Smith

Enter age:

-5

Invalid age

Sample Input/Output 3:

...

Enter patient name:

Bob Johnson

Enter age:

35

Enter phone number:

12345

Invalid phone number

Sample Input/Output 4:

Enter patient name:

Eve Brown

Enter age:

28

Enter phone number:

8765432109

Enter insurance ID:

@#$%7890

Invalid insurance ID

Sample Input/Output 5:

Enter patient name:

Michael Lee

Enter age:

50

Enter phone number:

9876543210

Enter insurance ID:

XYZ4567890

Enter doctor specialization:

Neurology

Doctor specialization not available

Sample Input/Output 6:

Enter patient name:

Sophia Garcia

Enter age:

40

Enter phone number:

8765432109

Enter insurance ID:

PQR1234567

Enter doctor specialization:

Dentistry

Doctor not available at specified time

Question on String Handling

1. PROBLEM STATEMENT

You are tasked with developing a program for analyzing a set of N DNA sequences provided as input. Each sequence consists of characters 'A', 'T', 'C', and 'G'. The program should determine the count of positions in each sequence where either all characters are purines (A or G) or all characters are pyrimidines (C or T). Purines and pyrimidines are nucleotide bases in DNA.

Constraints:

The number of sequences (N) should be greater than zero. If N is zero or negative, print "The number of sequences must be greater than zero" and terminate the program.

- All sequences must be of the same length. If they are of different lengths, print "All sequences are not of the same length" and terminate the program.

Input:

- The first line contains an integer N, the number of DNA sequences.

- The next N lines each contain one DNA sequence composed of characters 'A', 'T', 'C', and 'G'.

Output:

- N lines where each line contains two integers: the count of positions where all characters are purines and the count of positions where all characters are pyrimidines in the corresponding DNA sequence.

Sample Input / Output:

Example 1: Valid Input

Input:

3

ATCG

AGGG

CTTC

Output:

1 1

3 0

0 3

Explanation:

1. For the sequence "ATCG":

- Position 1: `A` (purine)

- Position 2: `T` (pyrimidine)

- Position 3: `C` (pyrimidine)

- Position 4: `G` (purine)

- Purine count: 1 (Position 4)

- Pyrimidine count: 1 (Position 3)

2. For the sequence "AGGG":

- Positions 1, 2, 3, 4: All are purines

- Purine count: 3

- Pyrimidine count: 0

3. For the sequence "CTTC":

-Positions 1, 2, 3, 4: All are pyrimidines

- Purine count: 0

- Pyrimidine count: 3

Example 2: Invalid Input (Different Length Sequences)

Input:

4

ATCG

AGG

CTTC

GCGT

Output:

All sequences are not of the same length

Explanation:

- The sequences have different lengths, so the program terminates with the error message.

Example 3: Invalid Input (Number of Sequences Less Than or Equal to

Zero)

Input:

0

Output:

The number of sequences must be greater than zero

Explanation:

- The number of sequences is zero, so the program terminates with the error message.

Example 4: Valid Input

Input:

2

AGCT

CGTA

Output:

1 1

0 2

...

Explanation:

1. For the sequence "AGCT":

- Position 1: `A` (purine)
- Position 2: `G` (purine)
- Position 3: `C` (pyrimidine)
- Position 4: `T` (pyrimidine)
- Purine count: 1 (Position 2)
- Pyrimidine count: 1 (Position 3)

2. For the sequence "CGTA":

- Position 1: `C` (pyrimidine)
- Position 2: `G` (purine)
- Position 3: `T` (pyrimidine)
- Position 4: `A` (purine)
- Purine count: 0
- Pyrimidine count: 2 (Positions 1 and 3)

Example 5: Invalid Input (Number of Sequences Less Than or Equal to Zero)

Input:

-1

Output:

The number of sequences must be greater than zero

Explanation:

- The number of sequences is negative, so the program terminates with the error message.

2. PROBLEM STATEMENT

You are developing a program to analyze the structure of N financial transaction identifiers provided as input. Each identifier is composed of a fixed number of alphanumeric characters. The program should identify positions in each identifier where all characters are digits ('0'-'9') or all characters are letters (either uppercase or lowercase). Finally, the program displays the count of such positions for each identifier.

Constraints:

- The number of identifiers (N) should be greater than zero. If N is zero or negative, print "The number of identifiers must be greater than zero" and terminate the program.

- All identifiers must be of the same length. If they differ in length, print "All identifiers are not of the same length" and terminate the program.

Input:

- The first line contains an integer N, the number of identifiers.

- The next N lines each contain one identifier composed of alphanumeric characters.

Output:

N lines where each line contains two integers: the count of positions where all characters are digits and the count of positions where all characters are letters (either all uppercase or all lowercase) in the corresponding identifier.

Sample Input/Output:

Sample Input 1:

4

A1B2C3D4

5G6H718J9

abcd1234

XYZW9876

Sample Output 1:

04

Explanation:

- Identifier 1 (A1B2C3D4): No positions have all digits; Positions 1-4 have all letters ('ABCD').

- Identifier 2 (5G6H718J9): No positions have all digits; Positions 2-5 have all letters ('GHIJ').

Identifier 3 (abcd1234): Positions 5-8 have all digits ('1234'); No positions have all letters.

- Identifier 4 (XYZW9876): Positions 5-8 have all digits ('9876'); Positions 1-4 have all letters ('XYZW').

Sample Input 2:

3

ABC123

DEF4567

GHIJKL

Sample Output 2:

00

Explanation:

- All identifiers are not of the same length (lengths are 6, 7, and 6 respectively), so the output is "All identifiers are not of the same length".

Sample Input 3:

-2

Sample Output 3:

...

The number of identifiers must be greater than zero

Explanation:

- The number of identifiers is less than or equal to zero, hence the output indicates the error message.

3. PROBLEM STATEMENT

A local library wants to implement a new system for book checkouts. To enhance security, they decide to require patrons to enter a passphrase that meets a specific criteria based on the books they want to borrow. The passphrase must consist of words where the sum of the alphabetical positions of the characters in the first half of each word equals the sum of the positions in the second half.

Criteria:

- For each word in the passphrase, split it into two halves. If the word has an odd length, ignore the middle character.

Calculate the sum of alphabetical positions (A=1, B=2, ..., Z=26) for each half.

If the sums of both halves are equal, the word is considered valid.

Otherwise, the word is considered invalid for the passphrase.

Write a program to implement this passphrase validation system. Your program should:

- Prompt the user to enter a passphrase containing multiple words separated by spaces.

- Validate each word according to the criteria.

Output whether each word in the passphrase is valid or invalid.

Constraints:

- Each word in the passphrase must consist only of uppercase or lowercase English letters.

- Ignore any words that contain digits, special characters, or whitespaces.

Example:

Input:

Enter the passphrase: hello noon zips

Output:

hello is not a valid word

noon is a valid word

zips is not a valid word

Explanation:

- "hello": Split into "hel" and "lo". Left sum = 8 (h = 8 e = 5 , Right sum = 15 (l = 12 , o = 15 . Not equal, so not valid.

- "noon": Split into "no" and "on". Left sum = 29 n = 14 o = 15 , Right sum = 29 (o = 15 , n = 14) . Equal, so valid.

- "zips": Contains characters other than letters, so it is not valid.

Your program should handle multiple words in the passphrase and provide clear output indicating which words meet the criteria.

=

4. PROBLEM STATEMENT

The library management system is launching a new online portal for book reservations. To ensure security and accuracy, users need to authenticate using a PIN generated based on their membership ID. Develop a Java application to automate the PIN generation process.

Note:

1. Start by verifying the length of the membership ID. It should be exactly 8 characters long; otherwise, print "<length of membership ID> is an invalid length" and terminate the application.

2. The membership ID should consist of 3 alphabets followed by 5 digits. If this format is not followed, print "<membership ID> is an invalid format" and terminate the program.

3. The first digit of the numeric part of the membership ID (following the alphabets) should not be '0'. If it is '0', print "<first digit> is an invalid digit" and terminate the program.

4. The last digit of the numeric part of the membership ID should correspond to the age category of the member: 1 for ages 10-20, 2 for ages 21-30, and so on up to 9 for

ages 80-90. If it doesn't match any category, print "<age category> is an invalid category" and terminate the program.

Logic of PIN Generation:

The PIN will be the concatenation of the even-positioned digits (0-indexed) in the numeric part of the membership ID.

Example: Membership ID - ABC12345, Age Category - 2

Generated PIN:

24

Sample Input / Output:

Sample Input 1:

Enter the membership ID

ABC12345

Sample Output 1:

PIN: 24

Sample Input 2:

Enter the membership ID

ABCD123

Sample Output 2:

7 is an invalid length

Sample Input 3:

Enter the membership ID

ABC01234

Sample Output 3:

0 is an invalid digit

Sample Input 4:

Enter the membership ID

ABZ12345

Sample Output 4:

ABZ12345 is an invalid format

Sample Input 5:

Enter the membership ID

ABC12356

Sample Output 5:

6 is an invalid category

5. PROBLEM STATEMENT

Students need to process a sentence where they are required to reverse the characters of each word that starts and ends with a vowel, while keeping other words unchanged. Help them implement this functionality using a Java program.

Requirements:

- If the sentence contains fewer than 2 words, print "Invalid sentence length".
- Words that start and end with vowels should have their characters reversed.
- Words should contain only alphabets and spaces; if not, print "<sentence> is an invalid sentence".

Sample Input/Output:

Sample Input/Output 1:

Enter the sentence

You are the apple of my eye

You are the elppa of my eye

Sample Input/Output 2:

Enter the sentence

Hello! How are you?

Hello! How are you? is an invalid sentence

Sample Input/Output 3:

Enter the sentence I love OpenAl's models

Invalid sentence length